Layout — monorepo arborescence, dependency rules, five-layer conventions

SPEC-ARCH-03 v0.2 OVERVIEW Jeremie Nunez

1 Purpose

Every architectural rule in this repo shows up as a *directory rule*. This spec lists the monorepo tree, the allowed dependency direction, and the two five-layer conventions (one for the backend, one for the frontend) so that a reviewer can tell at a glance whether a change respects the shape of the system.

2 Monorepo tree

```
apps/
  console/          React 18 + TanStack Router + r3f + sigma.js -
     operator UI
  console-api/      Fastify HTTP surface, SSA domain pack, container root
packages/
  cli/              Ink REPL + HTTP client to console-api (nine verbs)
  db-schema/        Drizzle ORM schema, pgvector columns, SQL migrations
  shared/           Cross-cutting types, enums, observability, frontend
     view builders
  sweep/            Kernel: audit loop + sim engine + ingestion scheduler
      (domain-agnostic)
  test-kit/         Test-only stubs, typed spies, fake ports, fixture
     parsing
  thalamus/         Kernel: cortices, planner/executor/reflexion, cycle
     loop (domain-agnostic)
```

Eight workspaces. The two runtime kernels are intended to stay domain-agnostic; SSA-owned policy lives in `apps/console-api/src/agent/ssa/`, while historical domain names that remain in shared seams or comments are documented as drift. `packages/test-kit` is a dev/test support package, not a runtime service seam.

3 Dependency direction

```
                   +---------- apps/console (React)
                   |              imports: shared, db-schema (types only
                      )
                   |
apps/console-api --+                          (terminal: nothing imports
    it)
  imports: thalamus, sweep, shared, db-schema
        |     |
        |     +---- packages/sweep (kernel)
        |             imports: thalamus, shared, db-schema
        |                          |
        +-------- packages/thalamus (kernel)
                    imports: shared, db-schema
                                |
                         packages/{shared, db-schema}  (leaves)

packages/cli  ----HTTP-first----> apps/console-api/**
  (HTTP for cycles / KG / why / KNN candidates; transitional public
```

```
   workspace imports remain for nano interpretation and a local accept
      command
   with no domain resolution handlers)

packages/test-kit ----test helpers----> packages/thalamus
```

Invariants.

- **No private bypass.** `packages/*` never imports from `apps/*`. Enforced by `.dependency-cruiser.js` at `pnpm arch:check:repo`.
- **One direction between kernels.** `packages/sweep` imports `packages/thalamus` (for `CortexRegistry` and `callNanoWithMode`, consumed by the sim turn runners). Not a leak: sim agents reuse the kernel's nano caller and skill registry. Reverse direction is forbidden.
- **CLI is a hybrid client.** `packages/cli` does not deep-import app TypeScript internals; cycle, KG, and why reads go through `ThalamusHttpClient`, while KNN candidate reads still use a direct fetch in `boot.ts`. The current boot path still uses public `@interview/thalamus` and `@interview/sweep` exports for nano interpretation and a local accept/review command. That local boot disables audit, promotion, and resolution handlers, so domain resolution remains an app-wired concern. The interpreter also reads the SSA skill file by path until that prompt is served through HTTP.
- **Agnosticity guard.** `scripts/arch-check-thalamus-agnostic.sh` strips comments and checks a narrow SSA vocabulary set in `packages/thalamus/src`; it is a Thalamus kernel guard, not a whole-repo or Sweep-domain scan.

4 Backend five-layer convention (`apps/console-api`)

Each feature lives as one file per layer. Controllers don't touch the DB; repositories don't speak zod; services own the logic.

```
routes/          Fastify route registrars; registerAllRoutes aggregator
   in routes/index.ts
controllers/     parse + validate via zod schemas, delegate to services
services/        application logic; orchestrates repos + thalamus /
   sweep packs
repositories/    Drizzle + Redis data access; repositories/queries/
   holds raw SQL
transformers/    row -> DTO shaping + request DTOs (*.dto.ts + *.
   transformer.ts)
schemas/         zod request-shape schemas consumed by controllers
agent/ssa/       SSA domain pack: skills, sources, sweep/sim port impls,
    followup, explorer
infra/           cross-cutting: health-snapshot.ts, sim-route-transport.
   ts
middleware/      auth, tracing, error-envelope
container.ts     composition root (see section 7)
server.ts        createApp / startServer entrypoint
```

Concrete trace — "Findings" feature.

Layer	File
Route	`apps/console-api/src/routes/findings.routes.ts` (`registerFindingsRoutes`)
Controller	`apps/console-api/src/controllers/findings.controller.ts`
Service	`apps/console-api/src/services/finding-view.service.ts`
Repository	`apps/console-api/src/repositories/finding.repository.ts` (Drizzle)
Transformer	`apps/console-api/src/transformers/finding-view.transformer.ts`, `finding-status.transformer.ts`

5 Frontend five-layer convention (`apps/console`)

Different stack, same discipline: one file per layer per feature, Context-per-port DIP.

```
src/main.tsx            bootstrap: buildDefaultAdapters() -> TanStack
   Router
src/routes/             file-based routes: __root, index, config, ops,
   sweep, thalamus
src/providers/          AppProviders nests Query + Api + Sse + Renderer +
    Propagator + Graph
src/features/           vertical slices: autonomy, config, ops, repl,
   sweep, thalamus
src/usecases/           React Query hooks (useCyclesQuery,
   useDecisionMutation, keys.ts)
src/hooks/              cross-cutting (useAnimatedNumber, useDraft,
   useTimeControl...)
src/adapters/           ports: api/, sse/, graph/, propagator/, renderer/
src/shared/ui/          AppShell, TopBar, Drawer, CommandPalette,
   CycleLoader, ErrorBoundary...
```

Concrete trace — "REPL" feature.

```
apps/console/src/features/repl/
  ReplContext.ts          createContext + useRepl()
  ReplProvider.tsx        state (turns, open, inFlight, sessionId);
                          calls postChatStream / postFollowUpStream from
                          @/adapters/sse/repl; dispatches to turnReducer
  reducer.ts              Turn, TurnAction, turnReducer, newTurn,
     TurnPhase
  types.ts                postTurn, isSlashCommand, TurnResponse
  ReplPanel.tsx           UI shell (input + transcript)
  TurnView.tsx            render one turn
  FollowUpPlanView.tsx    render the follow-up proposal list
  FollowUpTurnView.tsx    render an auto-launched follow-up turn
  ResultView.tsx          render a completed cycle result
  renderers/              per-step renderers (ChatRender, GraphTreeRender,
      PcEstimatorRender, ...)
```

Context-per-port DIP. Five ports, each with a `*Provider`, a default impl, and a hook: `ApiClientContext` / `ApiClient`, `SseClientContext` / `SseClient`, `RendererContext` / `RendererAdapter`, `PropagatorContext` / `PropagatorAdapter`, `GraphContext` / `GraphAdapter`. Features never touch `fetch` or `EventSource` directly.

6 Kernel packages

6.1 packages/thalamus — research kernel

```
src/
  cortices/       registry, executor, cortex-llm, guardrails, confidence,
                  field-correlation, strategies/{standard,strategist},
                     config, types
  services/       thalamus, planner, executor, reflexion, cycle-loop,
     research-graph,
                  finding-persister, stop-criteria
  explorer/       ONLY nano-caller.ts + nano-swarm.ts (scouts/curator
     moved to SSA pack)
  ports/          web-search, entity-catalog, source-fetcher, embedder (4
      ports)
  prompts/        agnostic builders: planner-generic, reflexion, nano-
     swarm, curator
  transports/     LLM chat transport + provider adapters (kimi, openai,
     minimax, local),
                  factory, fixture-transport, openai-web-search adapter
  config/         container.ts (buildThalamusContainer), runtime-config,
     register-runtime-config
  entities/       Noop / Null defaults co-located with port types
  repositories/   research-cycle, research-finding, research-edge
  transformers/   research.transformer
  types/          research.types
  utils/          ssrf-guard
```

6.2 packages/sweep — audit, sim, ingestion kernel

```
src/
  services/        nano-sweep, finding-router, messaging, sweep-
     resolution
  repositories/    sweep.repository.ts (Redis: sweep:suggestions:*)
  ports/           finding-routing, finding-schema, ingestion-registry,
                   nano-audit, promotion, resolution-handlers
  sim/             swarm, aggregator, turn-runner-sequential, turn-runner
     -dag,
                   memory, agent-builder, sim-orchestrator, promote,
                      perturbation,
                   schema (buildTurnResponseSchema), types, http/
    ports/         14 files: action-schema, aggregation-strategy, cortex-
       selector,
                   kind-guard, outcome-resolver, persona, perturbation-
                      pack, promotion,
                   prompt, queue, runtime-store, scenario-context,
                      subject, swarm-store
  jobs/            queues.ts (6 BullMQ queues), workers/ (5 workers +
     helpers),
                   ingestion-registry.ts, schedulers.ts (crons)
  config/          container.ts (buildSweepContainer), redis, register-
     runtime-config,
                   sim-fish-config
  transformers/, utils/, types/
```

Queues declared (`packages/sweep/src/jobs/queues.ts`): `sweep`, `audit-target`, `sim-turn`, `swarm-fish`, `swarm-aggregate`, `ingestion`. Matching workers under `jobs/workers/`. Cron schedules (`jobs/schedulers.ts`): `weekly-sweep`, `ingestion-tle-history` (6 h), `ingestion-space-weather` (04:30 daily), `ingestion-launch-manifest` (12 h), `ingestion-notams` (6 h).

7 Composition root

apps/console-api/src/container.ts is the one place where concrete adapters meet kernel ports.

```
buildContainer(config, logger, simTransport)
  |
  |-- wires Drizzle + Redis repositories
  |-- registerThalamusConfigDomains + registerSweepConfigDomains (
     runtime config)
  |-- pushes setters: setNanoConfigProvider, setPlannerConfigProvider,
  |                   setCortexConfigProvider, setReflexionConfigProvider
     ,
  |                   setSimFishConfigProvider
  |
  |-- buildThalamusContainer({
  |      db, skillsDir, dataProvider, domainConfig,
  |      webSearch, entityCatalog, sourceFetcher, embedder
  |    })                                   // <-- SSA adapters plug in
     here
  |
  |-- buildSweepContainer({
  |      redis, sim { personaComposer, promptRenderer, cortexSelector,
  |                   actionSchemaProvider, perturbationPack,
  |                   aggregationStrategy, kindGuard, telemetry/pc swarm
     launchers },
  |      ports { findingSchema, audit, promotion, findingRouting,
  |              resolutionHandlers, ingestion }
  |    })                                   // <-- same for SSA sweep +
     sim
  |
  +-- returns AppServices, consumed by routes/index.ts
```

SSA adapters come from apps/console-api/src/agent/ssa/: ssaFindingSchema, SsaPromotionAdapter, SsaAuditProvider, SsaFindingRoutingPolicy, createSsaResolutionRegistry, createSsaIngestionProvid SsaPersonaComposer, SsaPromptRenderer, SsaCortexSelector, SsaActionSchemaProvider, SsaPerturbatio SsaAggregationStrategy, SsaKindGuard, startTelemetrySwarm, startPcEstimatorSwarm.

8 SSA domain pack — apps/console-api/src/agent/ssa/

All domain-specific code lives here, not in packages/.

```
agent/ssa/
  domain-config.ts          the DomainConfig implementation (keywords,
     cortices,
                            mode instructions, fallback plan, daemon DAGs
                               )
  cortex-data-provider.ts   maps skill frontmatter sqlHelper names to
     Drizzle queries
  skills/                   markdown prompts (*.md) - the cortex "
     binaries"
  explorer/                 scout, crawler, curator, orchestrator (uses
     NanoSwarm)
  sources/                  per-source fetchers (celestrak, spectra, ...)
      + registry
  sweep/                    audit-provider, promotion, finding-routing,
     resolution-handlers
  sim/                      perturbation-pack, kind-guard, cortex-
     selector,
```

```
                              aggregation-strategy, prompt-renderer,
                                 persona-composer,
                              action-schema, swarms/{telemetry, pc-
                                 estimator}
  followup/                   policy + executor for the REPL FollowUpPlan
  ssa-entity-catalog.adapter.ts, ssa-source-fetcher.adapter.ts,
  ssa-voyage-embedder.adapter.ts, ssa-entity-extractor.ts
```

A different domain (threat-intel, pharma-vigilance) would be a sibling folder `agent/<pack>/` with the same shape — no kernel code changes.

9 Auxiliary packages

9.1 packages/shared

Types and cross-cutting only — no DB, no HTTP servers. `observability/steps.ts` holds the closed `StepName` registry (19 events) shared by the server pump and the client renderers. `ssa/` hosts *frontend-only* view builders (`conjunction-view`, `satellite-view`, `finding-view`, `kg-view`). Enums, zod schemas, logger, Prom metrics, pure utils.

9.2 packages/db-schema

17 Drizzle schema modules under `src/schema/` plus the `index.ts` barrel — `amateur-track`, `article`, `conjunction`, `exploration`, `fragmentation-event`, `itu-filing`, `launch`, `notam`, `research`, `satellite`, `sim`, `source`, `space-weather`, `sweep`, `tle-history`, `user`, `_vector`. 20 SQL migrations under `migrations/`; notable: `0001_hnsw_index.sql`, `0002_embedding_2048.sql`, `0003_sim_memory_hnsw` `0013_conjunction_knn_fn.sql`, `0014_satellite_embedding.sql`.

9.3 packages/cli

`bin: { ssa: "./src/index.ts" }`. `src/index.ts` shebangs into `src/boot.ts`. Router schema (`src/router/schema.ts`) exposes nine verbs: `query`, `telemetry`, `logs`, `graph`, `accept`, `explain`, `pc`, `candidates`, `clarify`. HTTP client `src/adapters/thalamus.http.ts` talks to `$CONSOLE_API_URL` (default `http://localhost:4000`) for cycle, KG, and why routes. The KNN candidate command currently performs a direct fetch in `boot.ts`, using `CONSOLE_API_URL` and bypassing the shared HTTP client/auth adapter. Local boot still wires a Sweep-backed accept/review command through public package exports, but audit providers, promotion, and resolution handlers are disabled there; telemetry and PC estimators are explicitly disabled until their CLI paths consume the console-api HTTP launch routes.

9.4 packages/test-kit

Test-only support package used by package and app tests. It exports typed spies, fake port helpers, fixture parsers, logger stubs, and Thalamus-facing nano / cortex stubs. It is allowed to depend on public test-facing Thalamus types, but it is not part of production wiring and should not appear in service-boundary arguments.

10 Drift notes (v0.1 → v0.2)

- The v0.1 layout was a nine-line listing. v0.2 makes the dependency rules explicit (§3) and traces both five-layer conventions (backend §4, frontend §5) with real file paths.

- v0.1 said "SSA skills live in `apps/console-api/src/agent/ssa/`". v0.2 documents the whole domain-pack surface: skills, sources, domain-config, cortex-data-provider, explorer, sweep port impls, sim port impls, followup, and four `ssa-*.adapter.ts` files (§8).
- The frontend five-layer (Context-per-port DIP) is promoted to a first-class section — v0.1 didn't mention it.
- The `sweep` → `thalamus` dependency is explicitly documented as intentional (sim turn runners consume kernel nano callers); v0.1 didn't mark direction rules.
- The CLI is documented as a hybrid client: HTTP for server-owned surfaces, public package imports only for the remaining local nano / resolution seams.
- Listings updated to reflect the actual state: six BullMQ queues, 14 sim ports, 17 schema modules plus a barrel, 20 migrations, 19 step events, and the test-only `test-kit` workspace.